

ML in Science Assignment 3

Hydraulic Cross Section

Group 15 - The Three Musketeers:

Vlad Chibulcutean (1780980)

Andreas Sinharoy (1804987)

Alex Gavrilu (1785060)

Part C:

Introduction

Our problem considers the laminar flow of a fluid - under the action of a pressure drop - through a porous medium, which extends along the interior of a channel. This problem is common in various engineering and geophysical applications, such as oil reservoir simulation, groundwater hydrology, and filtration processes. The accurate modelling of such flow systems is crucial for optimising performance and predicting behaviour under various conditions. Our problem inputs are cross-sections, represented by 40×40 images in which light pixels (with value 1) indicate locations where the fluid can flow, and closed pixels (with value 0) indicate locations that are impermeable to flow. That is, our input provides a representation of how much area the liquid manages to flow through, supported by the fact that these values of 0 or 1 are in m^2 . This binary representation also provides a clear and simplified view of the porous medium's structure, which is essential for ease of analysis and computation.

The desired output of our problem, in turn, is the corresponding hydraulic throughput S for any input image, which is in m^4 and is a metric independent of the fluid properties, channel length, or pressure drop. Instead, S is solely linked with the overall area of the section through which the fluid flows. This means that our target value should be computable by our model only using the input data that we are provided, thus focusing on the geometric and structural properties of the flow channels.

The data we received consists of three numpy arrays, which represent the inputs with which we train our model (*pub_input*), the outputs with which we train our model (*pub_out*), and the inputs that our model will ultimately be evaluated on (*pri_in*). *pub_input* and *pri_in* have shapes (1660, 40, 40) and (500, 40, 40), respectively, representing the facts that there are 1660 images (of 40×40 pixels) with which we should train our model, and 500 images of the same sizes with which we will evaluate our model. In contrast, *pub_out* has shape (1660,), thus just containing 1660 different unitary S values, each corresponding to one of the 1660 different images for model training. It is also important to note once again that in both input data sets, the values of the $40 \times 40 = 1600$ pixels are all either 1 or 0, representing whether fluid flows or does not flow through them.

Our task involves training a model to learn the relationship between the binary cross-sectional images and the hydraulic throughput S . This relationship is expected to be complex due to the varied and irregular structures of the porous media represented by the images. Traditional computational fluid dynamics approaches might be too computationally intensive for this task. Therefore, machine learning techniques, particularly convolutional neural networks (CNNs), are well-suited for this problem due to their ability to capture spatial patterns and features in image data.

Physical Dimensions:

As previously mentioned, the input values are in physical dimension m^2 , representing area, while the hydraulic throughput output is in m^4 , a measure that results from combining lengths in four dimensions, though not in a directly intuitive way like area (m^2) or volume (m^3). To achieve dimensional homogeneity, either the inputs would have to be manipulated into m^4 or the outputs into m^2 . Normally, we would consider both approaches, but this time we are directly asked to work on input data with physical dimension m^2 , and also told that our test submission will expect output data in m^2 too. Thus, it follows that the physical dimension of our output data will have to be changed to m^2 from m^4 .

There are a number of ways to do this. One way would simply involve square rooting the output data before training the model, as $\sqrt{m^4} = m^2$. This would achieve dimensional homogeneity and also create the desired physical dimension for the output. The simplicity of the method would also ensure minimum computational time or risks of implementation errors. Another simple method could be doing $\log_2(S)$, where S is the hydraulic throughput. This would once again be a simple method, easily computable, that would make the output data be in physical dimension m^2 . A perhaps more helpful method, however, could be dividing the hydraulic throughput output in m^4 by the sum of the values in the input. That is, for all values of either 0 or 1 in a given input 2d array, we would divide the corresponding output value by their sum, leading to a new output value to train the model on that is in m^2 . This combination of the input and output could provide some additional, helpful information to the model, potentially improving its performance, whilst also making the problem dimensionally homogeneous. Alternatively, the hydraulic throughput could also be divided by the area of the image, which is $1600 m^2$, leading to a new m^2 output value that would also be normalised, and ensuring that the network can predict a quantity based on the binary input.

Data Ranges

Input Analysis:

Maximum	Minimum	Range	Mean	Standard deviation	25th Percentile	Median	75th Percentile
1435	7	1428	384	339	57	202	626

Table 1 - Statistics for Number of Pixels Through which Liquids Pass (1's)

The provided statistics offer a comprehensive view of the distribution of liquid-permeable areas within the dataset of 40x40 images used as input for our neural network. The average number of passing cells, approximately 384, serves as a central measure of tendency. This average indicates that, on average, a significant portion of each image allows liquids to pass through, specifically about 24% of the total area.

This insight provides a baseline understanding of permeability across the dataset, which is crucial for contextualising the network's training and its input characteristics.

The range of passing cells, spanning from a minimum of 7 to a maximum of 1435, highlights the substantial variability in the dataset. This variability suggests that the images vary widely in their liquid-permeable areas, ranging from images with very limited passage points to those that are predominantly open. Such diversity challenges the neural network to learn robust patterns that can generalise across these different configurations.

The standard deviation, approximately 339, quantifies the dispersion of passing cells around the average. This measure indicates that the dataset exhibits significant variability in terms of liquid-permeable areas. A higher standard deviation implies that values are spread out over a wider range, reflecting the dataset's heterogeneity. The standard deviation value being similar in magnitude to the mean conveys the significant variability in the data.

The percentiles further elucidate the distribution of passing cells within the dataset. The median, which is 202.5, indicates that half of the images have 202 or fewer passing cells. This statistic is pivotal as it divides the dataset into two equal halves, providing a clear picture of the dataset's central tendency and helping to assess the distribution of liquid-permeable areas. Moreover, the 25th and 75th percentiles being at 57 and 626, respectively, convey that there are a notable number of cross-sections with very low permeability, as well as a significant number of cross-sections with more than 40% permeability.

In summary, these detailed statistics collectively provide a nuanced understanding of the dataset's composition and characteristics. In future implementation, they will be used to inform decisions regarding data preprocessing, model design, and performance evaluation, ensuring the neural network is effectively trained to handle the variability and nuances present in the 40x40 images.

Besides these statistics, we also looked at the number of cross-sections that shared the same number of white (passing) pixels, as well as the number of cross-sections that shared the same number of edges between black and white pixels. There were 1195 cross-sections that had at least one other cross-section with the same amount of white pixels, and

1641 cross-sections that had at least one other cross-section with the same amount of white and black edges, showing that there exist a significant number of similarities in between different network inputs. Moreover, there are as many as 23 distinct cross-sections with the same number of passing pixels, and 24 with the same number of white-black edges. These notable similarities between different inputs could simplify or complicate the computations of our neural network, depending on how much they convey about the corresponding hydraulic throughputs of these images. However, this aspect will be further analysed within the symmetries section.

Output Analysis:

Maximum	Minimum	Range	Mean	Standard deviation	25th Percentile	Median	75th Percentile
307825	7	307818	33201	67765	106	1627	24562

Table 2 - Statistics for Hydraulic Throughput Network Output

The statistics provided for the hydraulic throughput of the 40x40 cross-sections offer several insights into the distribution and variability of the data. The range, which is the difference between the maximum and minimum values, is exceptionally large at 307,818 indicating a substantial spread in the hydraulic throughput values. This large range suggests that the cross-sections can have drastically different levels of hydraulic throughput, with some sections allowing nearly complete flow and others almost none. The maximum value of 307,825 and the minimum value of 7 reinforce the wide disparity in throughput values.

A mean value of 33,201 indicates that, on average, the hydraulic throughput is relatively high. However, the mean alone does not convey the entire story due to the high standard deviation of 67,765. This high standard deviation signifies that the throughput values are widely spread around the mean, pointing to significant variability within the dataset.

The median value of 1,627 provides a contrasting perspective to the mean. Since the median is much lower than the mean, it indicates that more than half of the cross-sections have throughput values below 1,627. This suggests a positively skewed distribution, where a small number of

cross-sections with extremely high throughput values are pulling the mean upwards.

Examining the quartiles, the 25th percentile value is 106 and the 75th percentile is 24,562. These values indicate that 25% of the cross-sections have throughput values below 106, while 75% have values below 24,561. The significant difference between these percentiles further highlights the skewness in the data and suggests that a majority of the cross-sections have relatively low throughput values, with a few sections having extremely high throughput, thus creating a long tail in the distribution.

In summary, the dataset of hydraulic throughput values demonstrates a high degree of variability and skewness. Most cross-sections exhibit low to moderate throughput, while a few sections achieve exceptionally high values, significantly influencing the overall mean and contributing to the large range and standard deviation observed. Understanding this distribution is crucial for interpreting the performance and behaviour of these cross-sections in practical applications, as it highlights the potential for outliers and the importance of considering both central tendency and dispersion in analyses.

Symmetries:

For the symmetries, we considered a number of different transformations of the input data that could generate the exact same hydraulic throughput. Initially, we investigated translations, rotations, reflexions and permutations of the 40 by 40 cross-sections. Moreover, we considered the symmetry of changing cross-sections such that the number of edges between black and white nodes would remain the same.

To check permutations, we looked at different cross-section inputs that had the same number of white and black pixels, and then checked whether their corresponding hydraulic throughputs were the same. It didn't take long before we realised that this was not a valid symmetry.

```

def calc_whites(image):
    counter = 0
    for i in range(40):
        for j in range(40):
            if image[i][j] == 1:
                counter+=1
    return counter

whites_dict = {}

for i in range(1600):
    whites_dict[i] = []

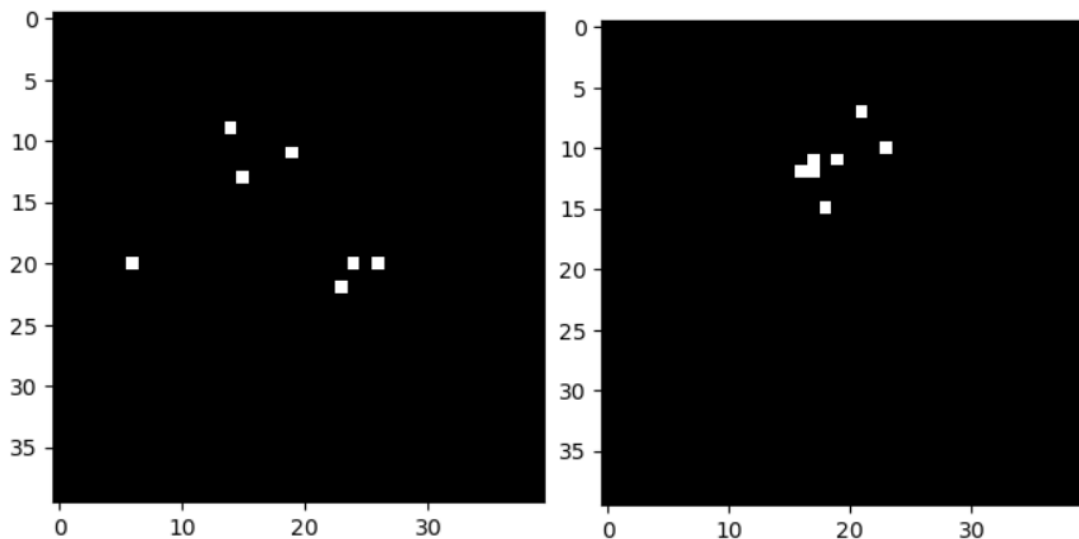
for i in range(1660):
    whites_dict[calc_whites(public_input[i])].append(i)

for i in range(1600):
    if not whites_dict[i]:
        del whites_dict[i]

```

Figure 1 - Code for Computing the Groups of Images with Equal Amounts of White Pixels (1's)

We used the above code to calculate all groups of inputs that had the same number of black and white pixels, and got counterexamples for this symmetry such as those at indices 125 and 458, shown below. These two cross-sections both have 7 total white pixels, but their hydraulic throughputs are at 7 and 8. 571, respectively. This shows that simply permuting the pixels around the image undoubtedly does not leave the hydraulic throughput unchanged.



Figures 2 & 3 - Image Representations of Inputs 125 and 458

We then checked for symmetry considering the same number of black and white edges, thinking that each edge would affect the throughput equally and

that thus permuting while keeping this constant would constitute a form of symmetry. However, we once again found that simply having the same amount of edges between black and white nodes was not sufficient, as exemplified by indices 474 and 543. These two inputs are shown below.

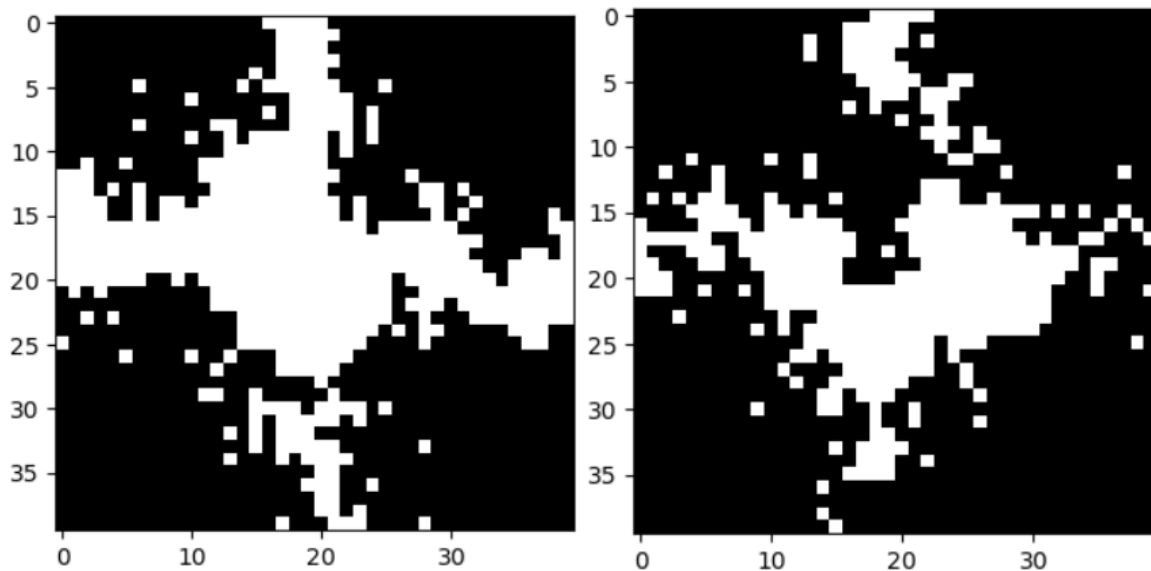


Figure 4 & 5 - Image Representations of Inputs 474 and 543

The left image (474) has a corresponding hydraulic throughput of 10498, while the right image has 4806, though both of them have 448 black-white edges. The code with which we calculated groups of images with the same amount of black-white edges can be found below:

```
def calc_bw_edges(image):
    counter = 0
    for i in range(40):
        for j in range(39):
            if (image[i][j] != image[i][j+1]):
                counter+=1

    for j in range(39):
        for i in range(40):
            if (image[i][j] != image[i][j+1]):
                counter+=1

    return counter

edges_dict = {}

for i in range(3120):
    edges_dict[i] = []

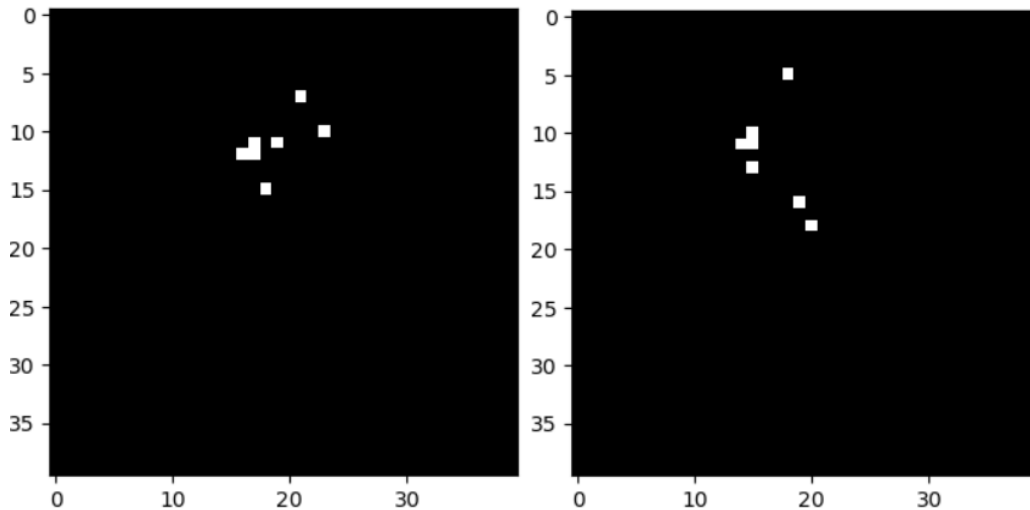
for i in range(1660):
    edges_dict[calc_bw_edges(public_input[i])].append(i)

for i in range(3120):
    if (not edges_dict[i] | len(edges_dict[i])<=1:
        | del edges_dict[i]

edges_dict
```

Figure 6 - Code for Computing the Groups of Images with Equal Amounts of Black-White Edges

Although our investigation into these two symmetries did not lead to usable results, it did help us realise which aspect of the input images do actually create symmetries. Specifically, we realised when analysing the images at indices 458 and 1231 (shown below), that as long as the same disjoint shapes (made up of white pixels) are present in different images, the hydraulic throughput corresponding to those images will be the same.



Figures 7 & 8 - Image Representations of Inputs 458 and 1231

These two cross-sections both have the exact same corresponding hydraulic throughput, at 8.57142.

When taking into consideration the physics of this problem, this idea also holds significance. The way in which water passes through gaps (given that the rate of incoming water is constant and uniform) is determined by three important factors: the size (area of the gap), the perimeter of the gap, and the shape of the gap. The two previous variants of this permutation theory (same number of white/black pixels and same number of edges between black and white pixels) respect two of these facts but ignore that of shape. The shape of the gaps/pores has a great effect on flow, as a long thin line gap of the same area as a circle gap will have a much lesser flow rate. This is influenced not just by the area but also by the flow resistance, which is higher in elongated shapes due to increased boundary interactions. According to the Hagen-Poiseuille law for laminar flow, the flow rate is proportional to the fourth power of the radius, highlighting the significant influence of shape. Hagen-Poiseuille law primarily is meant to describe the flow in pipes; however the use of radius still applies in

this context as the distance of a central point from the edges of the gap affects the hydraulic resistance on the water. The perimeter affects the amount of friction and drag exerted on the passing water, and a longer perimeter for the same area results in greater resistance. Additionally, the shape of the gap influences the streamline patterns of the fluid, potentially creating vortices and turbulence in irregular or elongated shapes. Thus, for a constant volume flow rate, gaps with circular or more compact shapes, which have lower hydraulic diameters as described by the hydraulic diameter formula, facilitate a higher flow rate compared to elongated or irregularly shaped gaps.

Given this conclusion, we are ready to specify the symmetry group of the problem, specifically analysing whether translations, reflexions, rotations, and the moving of the different shapes in the image would constitute symmetries.

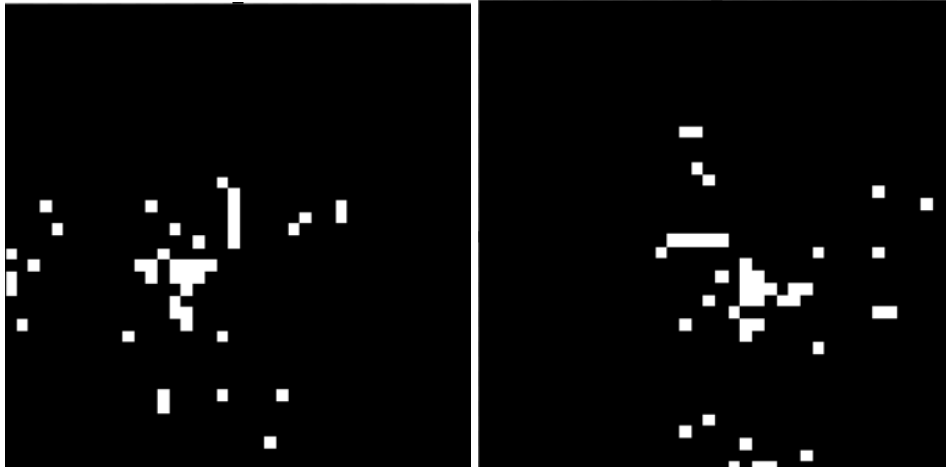
Part D:

Symmetry Group:

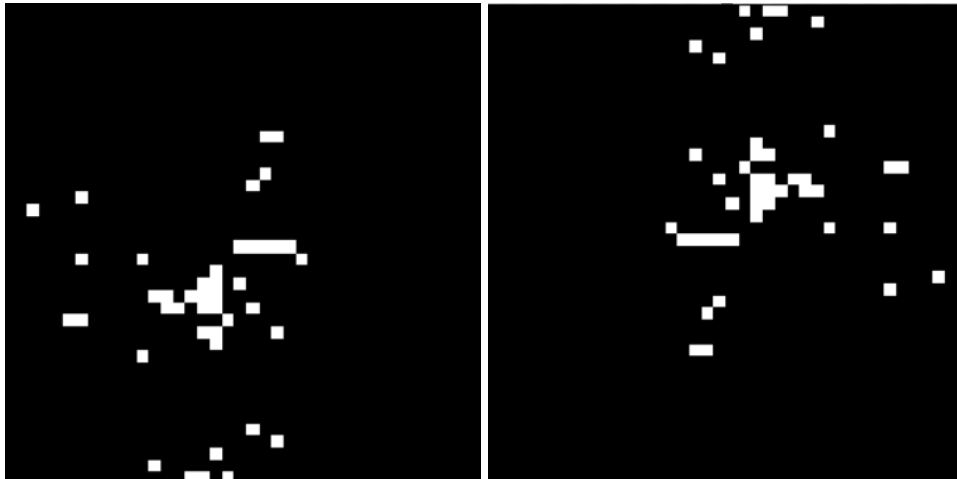
Following our above mentioned realisation, we immediately identified rotations and reflections as part of our problem's symmetry group. This is because both maintain the same disjoint groups of white pixels in the initial image versus the rotated/reflected image, and consequently, would have the same hydraulic throughput. Specifically, reflections in the x and y axes, as well as across the $y = x$ and $y = -x$ lines would be symmetrical, and would thus need to be considered for our model. As for rotations, these would be valid in 90, 180, and 270 degrees, leading to new images where the groups of white pixels are maintained.



***Figures 9 & 10 - Initial Image (left) and Image Rotated 90 degrees (right);
Both maintain the same disjoint groups of white pixels.***



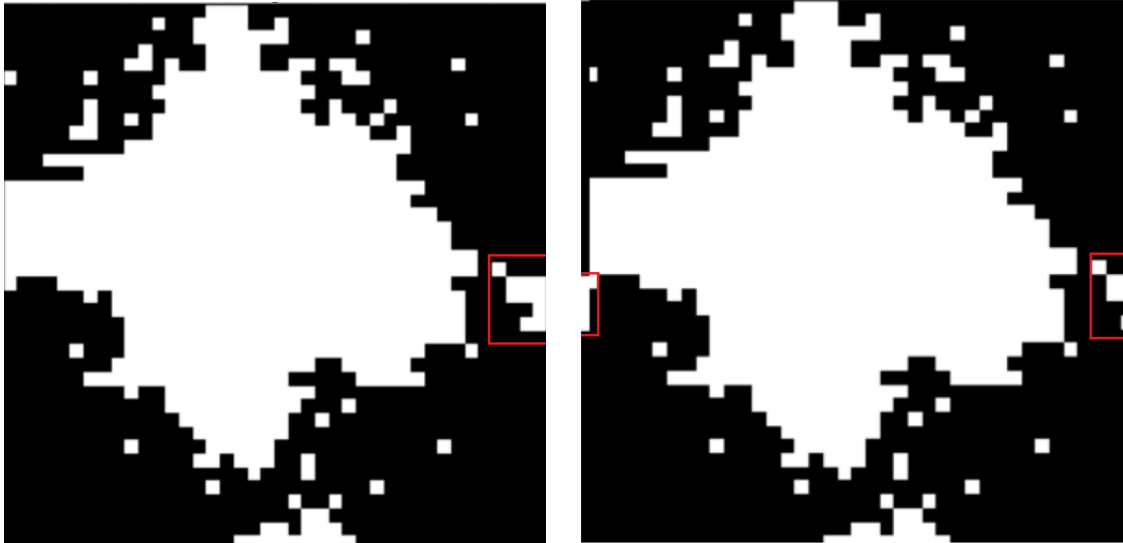
***Figures 11 & 12 - Initial Image Rotated by 180 degrees (right) and 270 degrees (left);
Disjoint groups also maintained.***



Figures 13 & 14 - Initial Image Reflected in Horizontal Axis (left) and Vertical Axis (right); Both maintain disjoint groups of white pixels.

Translations, on the other hand, do not satisfy this property. This is because as a pixel is translated outside of the image's edge and shows up on the opposite side, it could break up a pre-existing group of white pixels, and/or create a novel one. This would break our condition about the disjoint groups of white pixels remaining constant, and consequently, not satisfy the symmetry property. This breaking of the groups is evidenced by the two figures below, which show a horizontal translation of the left image to the right by one pixel. The group within the red square (in the left image) changes following the translation (in the right image), leading to the creation of two new disjoint groups (seen on the right of the right image), as well as an expansion of the large central group through the white pixels that changed side (as seen on the

left of the image). Thus, it follows that, in certain circumstances, translations will not maintain the symmetry of disjoint groups, and thus not keep the hydraulic throughput equal.



Figures 15 & 16 - Initial Image (left) and Initial Image Translated to the Right by One Pixel (right); Red square shows the changing of a group of white pixels.

At this point, our symmetry group thus consists of reflections and rotations. However, these do not make up all of the different permutations of the pixels in an image that maintain the same disjoint groups of whites. In reality, rotations and reflections account - on average - for a miniscule part of the complete set of permutations of an input that preserve this property. Consider, for example, the below image.

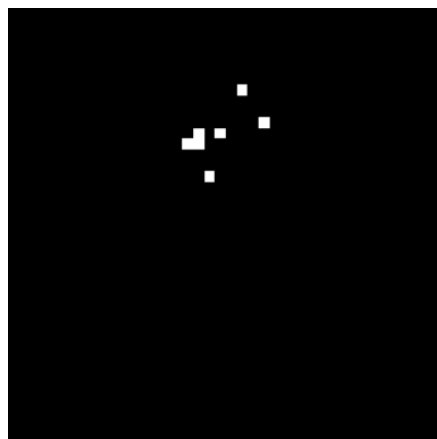


Figure 17 - Image Representation of Input at Index 458

Taking any one of the disjoint groups of white pixels in the above image, and moving it anywhere else in the image would, according to our current

hypothesis, preserve the hydraulic throughput of the cross-section. However, this dot can be moved to a significant number of different places, creating hundreds of different symmetrical images. Moreover, if we consider the concurrent moving of all of the groups, and all of the consequent, possible organisations of this set of disjoint groups in the 40 by 40 plane, we would undoubtedly get thousands of different symmetrical images for any one combination of disjoint groups of white pixels. This is only amplified by the fact that certain, more irregular shapes, can also be rotated while maintaining their structure and thus hydraulic throughput, leading to even more possible combinations. This fact easily shows how, under our current hypothesis, any given input can create a myriad of other symmetrical inputs. Moreover, these are not even close to being completely represented by the handful of inputs generated through reflections and rotations.

Having said this, we currently hold that any transformation of the input that will lead to the same number of disjoint groups of white pixels will preserve symmetry, and thus have the same corresponding hydraulic throughput. A small subset of this symmetry group consists of **rotations** and **reflections**, with the rest being described as **permutations of the input maintaining the disjoint, white groups**.

However, this hypothesis is based more on our observations of the training dataset rather than undeniable, physical facts. This is because the real world is not black and white, like our data, and two identical groups of white pixels may be white in our data while not actually contributing the exact same amount of hydraulic throughput (due to any combination of real-world factors). For instance, two unitary white pixels may be white in our dataset due to them having passed the threshold value making them white, but they may have passed it with different values, and thus contribute distinct values to the hydraulic throughput. That being said, such factors do not seem to be important in our training data, though we will be open to changing this conclusion if evidence to the contrary arises. This concludes our discussion of the symmetry groups.

Augmentation Strategy

Given any input, augmentation would involve generating all of its symmetrical inputs and also providing them to the model for training. For rotations, this would constitute simply flipping every input cross-section by 90, 180 and 270 degrees, and supplying all four sections to the model. For

reflections, every input image would have to be reflected in the x and y axes, as well as in the $y = x$ and $y = -x$ axes. Consequently, all 8 inputs would be provided to the model. Given a 2d array in Python, there exists built-in methods to carry out all of these transformations, such as *np. flipud* (horizontal reflection), *np. fliplr* (vertical reflection), *np. transpose* (diagonal reflection), *np. rot90* (90 degrees rotation), etc. The specific manner in which we implemented this method will be discussed later in the code explanation section.

The more difficult symmetry to augment are the permutations that preserve disjoint groups of white pixels. This is because, as mentioned previously, there are an enormous amount of ways in which most inputs can be permuted. The most straightforward, yet highly computationally-heavy method, would be brute forcing the augmentation, trying to move around and flip all groups in an input in all of the possible ways, and then feeding all of those to the model. However, creating a function that could do this would be extremely difficult, and its runtime would be substantial. Moreover, the augmentation would lead to such an increase in the number of inputs to the model that training would take excessively longer, ultimately hindering its efficiency.

Thus, the augmentation strategy would also require some level of optimisation. One such optimisation could involve not considering groups of white pixels with only a few pixels (such as one, two, or three), i.e., removing them from the input images. This is because their corresponding hydraulic throughput could be easily computed (for example, a group consisting of one white pixel has a corresponding hydraulic throughput of 1), and also because such small groups are responsible for the generation of a large number of symmetrical inputs. The latter is because a smaller group can be moved around to many more distinct locations than a larger group, and thus generate far more symmetrical inputs. This preprocessing strategy could help in lowering the model's workload post-augmentation by generating fewer inputs, but the model would still likely have a significantly-greater running time than without augmentation, which implies that augmentation may not be the best option for this problem.

Disambiguation Strategy

Disambiguation could be the better strategy for our problem given the large number of different inputs created through symmetries. This is because finding a specific image to map all symmetrical inputs to would monumentally

reduce the quantity of data that the model would have to work with, thus also reducing its runtime. However, the problem with disambiguation is defining this one input that all symmetrical inputs could map to. This is because the 40 by 40 cross-sections can vary greatly in the patterns of black and white pixels they exhibit. After long discussions and explorations of this topic, we ultimately concluded that there was no general rule through which we could map all symmetrical inputs for a given set of white pixel groups to one input since there was simply too much variety in the range of distributions of black and white pixels. We do not claim that such a method does not exist, but we could not come up with it.

We did, however, come up with a potential strategy that achieved the same end result. Firstly, we would extract the disjoint, white groups from any input image. From here, each group would undergo certain transformations (primarily rotations) such that equivalent groups could be identified, and each distinct group in a given input would be added to its personal 40 by 40 image, again in such a way as to ensure that equivalent groups would be represented in the same manner. Subsequently, all of the different images corresponding to different groups would be combined into one input image with many channels, each corresponding to a distinct group.

In this way, disambiguation could be carried out, leading to a much lower computational load for the model compared to augmentation due to much fewer inputs. However, the difficulty of implementation of this method would likely be rather high, as would its running time. Moreover, since the number of disjoint, white groups in an input can vary all the way from 0 (no groups) to 800 (groups of one pixel distributed equally), it follows that the number of channels for the inputs of the model would have to be 800, even when the number of actual groups is much lower. This would increase the model's computational load to a significant extent, potentially so much that this method may not even be desirable any longer. With these facts in mind, we considered disambiguation to be a perhaps better option than straight augmentation, yet still highly complicated to implement and computationally expensive.

Hardwiring

To hardwire our symmetries into the network architecture, we followed the method presented in the labs: (1) augmenting the input with all of the symmetries; (2) predicting different outputs based on the initial input and each

of the augmented inputs; (3) taking the average of all of the predicted outputs to then base the calculation of the gradient of descent on. To do so, we essentially alternated the architecture of the model for each symmetry. That is, for each symmetry and initial input, we calculated the symmetrical input, ran it through the model, and generated the predicted output. However, at the end of such a run of the model, we did not calculate the gradient of descent unless the model had already been run on all of the symmetrical inputs for our current initial input. Consequently, for n symmetries, the model would be run and a different output would be predicted n distinct times. Once n predicted outputs would be acquired, only then would the gradient of descent be calculated based on the average of those outputs.

For instance, consider the below case where there are no symmetries for the input. The training of the model on a given input would then only have the input pass through the following layers before calculating a gradient of descent:

Conv2d-86	[-1, 16, 40, 40]	160
LeakyReLU-87	[-1, 16, 40, 40]	0
Dropout-88	[-1, 16, 40, 40]	0
MaxPool2d-89	[-1, 16, 20, 20]	0
Conv2d-90	[-1, 16, 20, 20]	2,320
LeakyReLU-91	[-1, 16, 20, 20]	0
Dropout-92	[-1, 16, 20, 20]	0
MaxPool2d-93	[-1, 16, 10, 10]	0
Conv2d-94	[-1, 4, 10, 10]	580
LeakyReLU-95	[-1, 4, 10, 10]	0
Dropout-96	[-1, 4, 10, 10]	0
MaxPool2d-97	[-1, 4, 5, 5]	0
Flatten-98	[-1, 100]	0
LeakyReLU-99	[-1, 100]	0
Dropout-100	[-1, 100]	0
Linear-101	[-1, 1]	100

Figure 18 - Layers Involved in One Forward Pass (no symmetries)

However, if there exist, say, two distinct symmetries for the model's inputs, the training of the model on a given value (and its symmetrical values) would involve a greater number of layers. One subset of these layers is depicted in *Figure 18*, whilst the other two are shown below. Note that the two symmetries assumed in the image below are reflections in the x and y axes.

ApplyReflectionYSymmetry-18	[-1, 1, 40, 40]		ApplyReflectionXSymmetry-1	[-1, 1, 40, 40]	
Conv2d-19	[-1, 16, 40, 40]	160	Conv2d-2	[-1, 16, 40, 40]	160
LeakyReLU-20	[-1, 16, 40, 40]	0	LeakyReLU-3	[-1, 16, 40, 40]	0
Dropout-21	[-1, 16, 40, 40]	0	Dropout-4	[-1, 16, 40, 40]	0
MaxPool2d-22	[-1, 16, 20, 20]	0	MaxPool2d-5	[-1, 16, 20, 20]	0
Conv2d-23	[-1, 16, 20, 20]	2,320	Conv2d-6	[-1, 16, 20, 20]	2,320
LeakyReLU-24	[-1, 16, 20, 20]	0	LeakyReLU-7	[-1, 16, 20, 20]	0
Dropout-25	[-1, 16, 20, 20]	0	Dropout-8	[-1, 16, 20, 20]	0
MaxPool2d-26	[-1, 16, 10, 10]	0	MaxPool2d-9	[-1, 16, 10, 10]	0
Conv2d-27	[-1, 4, 10, 10]	580	Conv2d-10	[-1, 4, 10, 10]	580
LeakyReLU-28	[-1, 4, 10, 10]	0	LeakyReLU-11	[-1, 4, 10, 10]	0
Dropout-29	[-1, 4, 10, 10]	0	Dropout-12	[-1, 4, 10, 10]	0
MaxPool2d-30	[-1, 4, 5, 5]	0	MaxPool2d-13	[-1, 4, 5, 5]	0
Flatten-31	[-1, 100]	0	Flatten-14	[-1, 100]	0
LeakyReLU-32	[-1, 100]	0	LeakyReLU-15	[-1, 100]	0
Dropout-33	[-1, 100]	0	Dropout-16	[-1, 100]	0
Linear-34	[-1, 1]	100	Linear-17	[-1, 1]	100

Figures 19 & 20 - Layers Involved in Forward Passes of Symmetrical Inputs; Initial Input Reflected in y-axis (left) and Initial Input Reflected in x-axis (right)

Thus, the gradient of descent for a set of inputs with two symmetries would only be computed after passing the initial input through the three series of layers shown in *Figures 18, 19* and *20*. Importantly, this would be done in a sequential manner, with each of the three series of layers taking the initial input and producing a distinct output.

This hardwiring strategy has both its benefits and drawbacks. On the one hand, it ensures that the network explicitly incorporates symmetries by augmenting the input data and processing each augmented version separately. Moreover, by averaging the predictions from these augmented inputs before calculating the gradient, the model effectively learns to generalise across symmetrical transformations of the input.

On the other hand, this approach increases computational complexity and training time since each symmetry requires separate computations through the network. Moreover, it assumes prior knowledge of the symmetries and may not generalise well to novel symmetries or variations in real-world data.

Regardless of its drawbacks, this method elegantly incorporates input symmetries into the model's architecture and can be implemented relatively easily, unlike disambiguation, thus making it a highly valuable strategy. Ultimately, this is the method that we decided to go forward with.

Part E:

Code Overview and Explanation

To start off, the below code imports various libraries essential for data manipulation. *numpy* (*np*) is used for numerical operations and arrays, while *floor* from *math* rounds numbers down. *matplotlib.pyplot* (*plt*) handles plotting

and visualisation. The *torch* library is the core of PyTorch, supporting tensor operations and deep learning, with *torch.nn* and *torch.nn.functional* providing neural network layers and functions. *pandas* (*pd*) is for data manipulation and analysis. *torch.utils.data*'s *DataLoader* and *Dataset* manage data loading and batching. *tqdm* adds progress bars to loops. *torchsummary*'s *summary* function outlines model architecture. *copy* allows deep copying of objects, and *train_test_split* from *sklearn.model_selection* splits data into training and testing sets.

```
# Imports used throughout the code
import numpy as np
from math import floor
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
import pandas as pd
from torch.utils.data import DataLoader, Dataset
from tqdm import tqdm
from torchsummary import summary
import copy
from sklearn.model_selection import train_test_split
```

Subsequently, we import the three provided datasets through the method *np.load*, storing each of them in a distinct numpy array.

```
# Load the 3 .npy files
file_path1 = 'pub_out.npy'
public_output = np.load(file_path1)
file_path2 = 'pub_input.npy'
public_input = np.load(file_path2)
file_path3 = 'pri_in.npy'
private_input = np.load(file_path3)
```

Next, we square root the output values, since we were asked to train the model on output values in m^2 , and our initial output values were in m^4 . Subsequently, we convert the public inputs and outputs to arrays of values of type float32 to ensure that our future computations with these values are defined accordingly. Moreover, we set the device that we will be training the model on

to be cuda if this is available (we made sure that cuda was available when actually training), and the cpu if cuda is not available.

```
#This square roots each output
public_output = np.sqrt(public_output)

# Convert labels to a NumPy array if they are not already
public_input = np.array(public_input, dtype=np.float32)
public_output = np.array(public_output, dtype=np.float32)

# Set device to train the model on
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Next, we carried out a training/testing data split in proportion 0.8/0.2 in order to be able to train and validate our model with distinct data. To this end, we made use of the *train_test_split* method from the *sklearn.model_selection* library.

```
# Define train and test dataset
x_train, x_test, y_train, y_test = train_test_split(
    public_input, public_output, test_size=0.2, random_state=42
)
```

The next step for us was defining the *df_to_tensors* function, which converts input data (*x_test* and *y_test*) from pandas DataFrame format into PyTorch tensors. It first transforms *x_test* and *y_test* into tensors of type float32. The *.unsqueeze(1)* method adds an extra dimension to each tensor, which is often required for compatibility with neural network layers expecting inputs with a specific shape. Finally, the tensors are moved to the specified device (e.g., CPU or GPU) using *.to(device)*. The function returns the processed train_data and labels tensors, ready for use in model training or evaluation.

After this, we used the *PandasData* class, a custom dataset for holding tensors needed to train a neural network that inherits from PyTorch's *Dataset* class. In the *__init__* method, it converts the input data (*x_train* and *y_train*) from pandas DataFrame format into PyTorch tensors of type float32, and adds an extra dimension with *.unsqueeze(1)*. The *__len__* method returns the size of the dataset, allowing PyTorch to know the number of samples. The *__getitem__* method retrieves the input tensor and corresponding label tensor for a given index, facilitating data access during training.

```

# Define train and test dataset
def df_to_tensors(x_test, y_test):
    train_data = tensor(x_test, dtype=float32).unsqueeze(1).to(device)
    labels = tensor(y_test, dtype=float32).unsqueeze(1).to(device)
    return train_data, labels

# Define a DataSet type based on the previous one to hold the tensors
of input needed to train the nn
class PandasData(Dataset):
    def __init__(self, x_train, y_train):
        self.input = tensor(x_train,
dtype=float32).unsqueeze(1).to(device)
        self.labels = tensor(y_train,
dtype=float32).unsqueeze(1).to(device)

    def __len__(self):
        return self.input.size()[0]

    def __getitem__(self, index):
        return self.input[index], self.labels[index]

```

As shown below, the code then calls the method *df_to_tensors* described above, and creates a dictionary holding the training data and testing data that is later used for the model's training.

```

# Initialise the dataset with the training and testing data
test_input, test_labels = df_to_tensors(x_test, y_test)
data_dict = {
    "train": PandasData(x_train, y_train),
    "x_test": test_input,
    "y_test": test_labels,
}

```

Below several classes are then defined that contain forward pass methods that return some symmetrical input. These are all the symmetries that the architecture will later hardwire into itself. The symmetries that we define classes for are:

- Reflections of the image in the horizontal(x) axis
- Reflections of the image in the vertical(y) axis
- Reflections of the image in the bottom left to top right diagonal (line $y = x$)

- Reflections of the image in the top left to bottom right diagonal (line $y = 40 - x$)
- Rotations of the image 90° about the centre of the image
- Rotations of the image 180° about the centre of the image
- Rotations of the image 270° about the centre of the image

All these classes use the tensor form of the data as its input so the process of performing the transformations described can be done very simply in one line as tensors have many useful functions that can manipulate their structure. We do note that these are not all of the existing symmetries of our inputs. However, due to time constraints and the computational load on the model, these are the symmetries that we did end up considering. For information on how we would have implemented the more complex symmetries (described earlier in this report), see our sections on symmetry under parts C and D.

```
# Apply reflection over x-axis
class ApplyReflectionXSymmetry(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
        # Apply reflection over x-axis
        x_reflect_x = x.flip(2)
        return x_reflect_x

# Apply reflection over y-axis
class ApplyReflectionYSymmetry(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
        # Apply reflection over y-axis
        x_reflect_y = x.flip(3)
        return x_reflect_y

# Apply reflection over main diagonal
class ApplyReflectionMainDiagonal(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
```

```

        # Apply reflection over the main diagonal (transpose the last
        # two dimensions and then flip along the horizontal axis to m
        # match the diagonal reflection)
        x_reflect_main_diag = x.transpose(2, 3)
        return x_reflect_main_diag

# Apply reflection over anti diagonal
class ApplyReflectionAntiDiagonal(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
        # Apply reflection over the anti-diagonal
        # First, flip the matrix along both the horizontal and vertical
        # axes, and then transpose the last two dimensions to match the
        # anti-diagonal reflection
        x_reflect_anti_diag = x.flip(2).flip(3).transpose(2, 3)
        return x_reflect_anti_diag

# Apply Rotation of 90
class ApplyRotation90Symmetry(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
        # Rotate tensor by 90 degrees
        x_rot_90 = torch.rot90(x, k=1, dims=(2, 3))
        return x_rot_90

# Apply Rotation of 180
class ApplyRotation180Symmetry(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
        # Rotate tensor by 180 degrees
        x_rot_180 = torch.rot90(x, k=2, dims=(2, 3))
        return x_rot_180

# Apply Rotation of 270
class ApplyRotation270Symmetry(nn.Module):
    def __init__(self):
        super().__init__()

```

```
def forward(self, x):
    # Rotate tensor by 270 degrees
    x_rot_270 = torch.rot90(x, k=3, dims=(2, 3))
    return x_rot_270
```

The `SymmetricModelWrapper()` class below wraps around an existing model (`self.model`) and incorporates a series of symmetry transformations via `nn.ModuleList`, including reflections and rotations, applied to input data (`x`). During the forward pass, each transformed input undergoes prediction with `self.model`, and their outputs are averaged to yield a final prediction, improving the model's ability to generalise across different orientations of the input data.

`create_symmetric_model()` defines a straightforward CNN using `nn.Sequential()`, consisting of convolutional layers followed by ReLU activations and average pooling, finishing with fully connected layers for feature analysis and prediction.

```
# Symmetry combining class
class SymmetricModelWrapper(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model
        self.augmentations = nn.ModuleList([
            ApplyReflectionXSymmetry(),
            ApplyReflectionYSymmetry(),
            ApplyReflectionMainDiagonal(),
            ApplyReflectionAntiDiagonal(),
            ApplyRotation90Symmetry(),
            ApplyRotation180Symmetry(),
            ApplyRotation270Symmetry(),
        ])
    # Forward pass for symmetrical input variants
    def forward(self, x):
        outputs = [self.model(aug(x)) for aug in self.augmentations]
        outputs.append(self.model(x)) # Include the original input
        outputs = torch.stack(outputs, dim=0)
        return outputs.mean(dim=0)
```

Should be the same as the naive model with the exception of the addition of our two new layers.

```
def create_symmetric_model():
    return nn.Sequential(
        # Layer1
        nn.Conv2d(1, 16, kernel_size=(3, 3), padding=(1, 1)),
        nn.ReLU(),
        #nn.Dropout(p=0.1), # Add dropout layer
        nn.AvgPool2d(kernel_size=(2, 2)),

        # Layer2
        nn.Conv2d(16, 32, kernel_size=(3, 3), padding=(1, 1)),
        nn.ReLU(),
        # nn.Dropout(p=0.5), # Add dropout layer
        nn.AvgPool2d(kernel_size=(2, 2)),

        # Layer3
        nn.Conv2d(32, 64, kernel_size=(3, 3), padding=(1, 1)),
        nn.ReLU(),
        # nn.Dropout(p=0.5), # Add dropout layer
        nn.AvgPool2d(kernel_size=(2, 2)),

        # Layer4 (no avg)
        nn.Conv2d(64, 64, kernel_size=(3, 3), padding=(1, 1)),
        nn.ReLU(),

        # Layer5 (no avg)
        nn.Conv2d(64, 64, kernel_size=(3, 3), padding=(1, 1)),
        nn.ReLU(),

        # Layer6
        nn.Conv2d(64, 32, kernel_size=(3, 3), padding=(1, 1)),
        nn.ReLU(),
        # nn.Dropout(p=0.5), # Add dropout layer
        nn.AvgPool2d(kernel_size=(2, 2)),

        # Layer7
        nn.Conv2d(32, 16, kernel_size=(3, 3), padding=(1, 1)),
        nn.ReLU(),
        # nn.Dropout(p=0.5), # Add dropout layer

        # input flatten before final 2 layers
        nn.Flatten(start_dim=1),
```



```

    # Layer8 (no avg)
    nn.ReLU(),
    nn.Linear(16*2*2, 32, bias=False),

    # Layer9 (no avg)
    nn.ReLU(),
    nn.Linear(32, 1, bias=False),
)

```

The function below calculates the Root Mean Squared Percentage Error by computing the square root of the mean squared differences between the true and predicted values, normalised by the true values.

```

# Root mean square percentage error function for training
def rmspe(y_pred, y_true):
    return ((y_true - y_pred) / y_true).pow(2).mean().sqrt()

```

Below is the training method for our neural network. We take as parameters: the previously initialised dataset, the first initialised model, the number of epochs for the total training, the learning rate, batch_size, a value to indicate when to print loss data, and a value to indicate when to prompt to end the training and return the current best model.

This method per epoch does the following steps:

- First the training batches of data are taken and used to train and improve the model for one epoch. The error for the training data is then calculated, and then that error is used when calculating the new weights
- The gradient is then zeroed out to be refreshed (optimizer.zero_grad()), and then backpropagation is done and the new weights based on the error are assigned (loss.backward(), optimizer.step()). The training error is then appended to the dictionary of loss
- Next the model after this round of batch training uses the model to predict the values of a test batch. The error for the testing data is then calculated and appended to the loss dictionary
- Then several checks and prints are done by the method
 - First the model checks whether the current best_model is the best in terms of test error loss, and if it is not then the best model is changed and the best test loss is changed

- Then the various losses at that point in time are printed to be see epoch results
- After this the method checks the current epoch and sees whether it needs to stop to ask to end the training on this epoch. if n is entered then training ends and the current model, best model and loss dictionary are returned. If y is entered the training continues

The method once complete returns the latest trained model, the model with the lowest test error and the loss dictionary for all the epochs. All data is also moved back to the cpu so no data address errors occur.

```
def train_model2(
    data, model, nepochs=1, lr=1e-6, batch_size=1, print_every=1,
    prompt_every=200
):
    # First move all the data and the model to the GPU for processing
    model.to(device)
    x_test = data["x_test"].to(device)
    y_test = data["y_test"].to(device)

    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    loss_dict = {"train": [], "test": []}

    num_batches = floor(len(data["train"]) / batch_size)

    # Initiate best model for saving
    best_model = None
    best_test_loss = float('inf')

    # Begin the per epoch training loop
    for epoch in tqdm(range(nepochs)):
        epoch_loss_sum = 0

        # First train the model on the training batches
        for x_batch, y_batch in DataLoader(
            data["train"], batch_size=batch_size, shuffle=True
        ):
            x_batch, y_batch = x_batch.to(device), y_batch.to(device)
            y_pred = model(x_batch)
            # now calculate the test loss and adjust weights accordingly
            loss = rmspe(y_pred, y_batch)
```

```

        epoch_loss_sum += loss.item()
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # Add training to loss dict
    train_loss = epoch_loss_sum / num_batches
    loss_dict["train"].append(train_loss)

    # Calculate error for test batch
    with torch.no_grad():
        y_pred = model(x_test)
        test_loss = rmspe(y_pred, y_test)
        loss_dict["test"].append(test_loss.item())

    # Check if current model is better than best via test error
    if test_loss.item() < best_test_loss:
        best_test_loss = test_loss.item()
        best_model = copy.deepcopy(model)

    # Print out error info for this epoch
    if (epoch + 1) % print_every == 0:
        print(f"Epoch {epoch + 1} - Train Loss: {train_loss},
Test Loss: {test_loss.item()}, Best Test Loss: {best_test_loss}")

    # Check if epoch is on stopping point
    if (epoch + 1) % prompt_every == 0:
        cont = input(f"Continue training after epoch {epoch +
1}? (y/n): ").strip().lower()
        if cont != 'y':
            print("Stopping training.")
            best_model.to('cpu')
            model.to('cpu')
            return best_model, model, loss_dict

    # Move model and data to CPU from GPU
    best_model.to('cpu')
    model.to('cpu')
    return model, best_model, loss_dict

def rmspe(y_pred, y_true):
    return torch.sqrt(torch.mean(((y_true - y_pred) / y_true) ** 2))

```

The symmetry for the model, used to apply our input symmetries in the network, is instantiated using the previously mentioned `SymmetricModelWrapper` class.

The models are also moved to be used on the GPU and the summary object is also instantiated and moved to the gpu.

The training is then instantiated in and performed to return the 2 different models, `best_symmetrical_model` and `symmetrical_model`. We also specify the specific parameters of our training in the brackets of this instance.

```
symmetric_model_initiator =  
SymmetricModelWrapper(create_symmetric_model())  
# Print the summary of the model  
  
# Move the model to the GPU  
symmetric_model_initiator.to(device)  
  
# Ensure all parameters are on the GPU  
symmetric_model_initiator = symmetric_model_initiator.to(device)  
  
# Print the summary of the model  
summary(symmetric_model_initiator, (1, 40, 40), device=str(device))  
  
# Initialise model and begin training  
symmetric_model, best_symmetric_model, symmetric_loss = train_model2(  
    data_dict,  
    SymmetricModelWrapper(create_symmetric_model()),  
    nepochs=10000,  
    lr=0.001,  
    batch_size=16,  
    print_every=20,  
    prompt_every=1000  
)
```

Below the code saves the `best_symmetrical_model` saved from the most recent training performed. The `private_input` needed to be used for the kaggle submission is loaded into a numpy array and from there it is pre-processed in the same way as the training and validation data. Placed into an array of 2d-arrays, which is then made into tensors for use in the `best_symmetrical_model`. The data is then used to predict values using the model. These predicted values are then placed in a dataframe of the exact

format needed on kaggle and the data frame is saved as a csv file to be uploaded.

```
# Save best model
torch.save(best_symmetric_model, 'model.pth')
model = torch.load('model.pth')

# Process private kaggle data for model
x_data_test = np.load('pri_in.npy')
x_data_test = x_data_test.reshape(-1, 1, 40, 40).astype(np.float32)
x_data_test = torch.from_numpy(x_data_test).float()
model = torch.load('model.pth')

# Run model on data
with torch.no_grad():
    prediction = model(x_data_test).flatten().numpy()

# Make id list
ids = range(0, len(prediction))

# Add to a df
df_predictions = pd.DataFrame({
    "ID": ids,
    "Solution": prediction
})

# Export df as csv
df_predictions.to_csv('predictions2valid.csv', index=False)
```

Architectural Choices

The model begins with a sequence of convolutional layers followed by ReLU activations. Each convolutional layer processes the input data through a set of trainable filters that slide over the input's dimensions to extract different features such as edges, textures, and patterns in the data. The `input_channel` parameter refers to the number of channels (or depth) of the input data, which corresponds to the number of feature maps provided by the previous layer (1 for our input as it is just one singular sequence of images). The `output_channel` parameter specifies the number of filters that the convolutional layer will apply, thus determining the number of feature maps or the depth of the output data. By

stacking multiple convolutional layers, CNNs can learn increasingly abstract and complex representations of the input.

We experimented with the number of convolutional layers in our model as well as the number of filters (output_channels) each layer would apply. Some exploration we did included increasing and then decreasing the number of filters. For instance, first layer applying 16 filters, second 32, third 16, and fourth 8. We then compared this with keeping the number of filters stagnant over each layer. For instance, each layer applies 64 filters to its input. After extensive comparisons and experimentation, we discovered there was very negligible difference between each method for our model, as the accuracy the models would plateau at were relatively similar regardless of how we changed the composition of filters in that way. Ultimately, we settled for a combination of the two, where we would increase the number of filters in each layer until we would get to a certain number, say 64, then we would have 2-3 layers which would not alter this number, and then finish by slowly lowering the number of filters in the last layers. Specifically, our best model had the following distribution of number of filters: 16 – 32 – 64 – 64 – 64 – 32 – 16.

We also explored how the number of filters affect the performance. We ran models with a relatively low number, such as on average, 8 filters per layer. We also ran models with a high number of filters on average, such as a model which contained 128 filters per layer. Generally speaking, increasing the number of filters increased the speed at which the model improved. However, if the number of filters was set too large, the model would begin to plateau at a lower accuracy or begin to overfit the data. After much experimentation we discovered that having each layer contain a range of 1 to 64 filters provided the best results.

Finally, we experimented with how the number of convolutional layers affected the model's performance. For instance, we had models that contained only 2 layers, while others that contained 4 or even 5 layers. The results were that the more layers we added, the faster and more accurate our models would become. However, a common occurrence with models with a large number of convolutional layers was that they tended to become overfitted. For instance, our training loss would decrease consistently to highly desirable accuracies, however, the testing loss would not decrease in the same manner. This is an indication that the model would learn the training dataset to an extremely high

degree, but would not be able to apply this effectively when predicting new inputs encountered (such as in the testing dataset). A reason for this, we assumed, was that after a certain point, the added abstractions of our input did not add any relative information for the model to use to base the throughput of the image on. This resulted in our model becoming even less effective while using a higher computational load. We ended up therefore using 4 layers, which we empirically discovered provided an adequate abstraction of the images with high accuracies, while not being too computationally heavy or leading to overfitting.

For some of our models we experimented with utilising Leaky ReLU layers as opposed to ReLU activation. The reasoning for this is that they help with preventing the issue of dead neurons and stagnating accuracies during training by allowing a small, non-zero gradient when the unit is not active. For certain training sessions, we encountered our accuracy plateauing below that of which we wanted. We therefore thought adding these Leaky ReLU layers would help counteract this. Ultimately, this did not end up being the case, with its implementation rather decreasing the accuracy of our models. Possible explanations could include improper initialization of the leaky ReLU layers, improper parameter setting, or perhaps standard ReLU or other activation functions might provide better performance by more effectively capturing the nonlinear relationships within the image data. Regardless, we ended up using standard ReLU functions as our activation across the model.

Almost every time a convolutional layer and ReLU layer is applied in our model, it is then followed by an average pooling layer to reduce the dimensions of the input, helping with computational efficiency and feature abstraction. The way average pooling for our model works is by having a 2 dimensional kernel that iterates through subsequent sections of the input data, calculates the average value in each section, and then compresses the data so that the output holds just the average values of each of these sections. The scale of which the data is compressed is determined by the size of the kernel. For our model we set the kernel size to be (2, 2), hence cutting the dimensions of each 2 dimensional input of the pooling layer by half (e.g. (10, 10) input gets compressed to (5, 5) output). The reason these layers are important is because it allows the model to understand and process the high dimensional inputs of images in lower dimensions. Thus, providing an abstraction of the image that the computer can process effectively to a one dimensional output such as throughput value.

We also experimented with the number of average pooling layers in our model. This resulted in us needing to find the proper compression of the images suitable for the linear layer to map them to a throughput value. We experimented with this thoroughly, and compared results when the images were compressed to dimensions of (10, 10), (5, 5), (2, 2), and (1, 1). In general, we found that compressing to dimensions of (5, 5) or (2, 2) resulted in the most accurate models. The reason for this being when the dimensions are too high (i.e (10, 10)), the linear layer struggles to map such a complex input accurately to the throughput, and when it's too low (i.e (1, 1)), the output has too few dimensions to properly represent all the complexities of the (40, 40) images.

In some of our models, after the first convolutional layer, we added a dropout layer with p set to 0.1. This layer was sometimes beneficial because it helped prevent overfitting by randomly deactivating a fraction of neurons (in this case 0.1 neurons) during each training iteration. However, when we created models which we knew did not suffer from overfitting (as the model was not overly complex, and the training loss and testing loss decreased steadily together), removing the dropout layers helped improve the accuracy. Thus, in our final model dropout layers were consciously removed.

After the final convolutional layer, there is a flattening layer which flattens its output. The reason for this being that the dimension of the output of the final convolutional layer will be in the form (batch_size, nr_output_channels, image_abstraction, image_abstraction). The next layer, which is a linear layer, will expect a 2 dimensional input. Therefore, the output will be flattened to (batch_size, nr_output_channels * image_abstraction * image_abstraction), satisfying the input requirements for the subsequent layer.

The flattened feature maps are then processed through fully connected layers with ReLU activations. These layers allow the model to learn complex relationships between the extracted features and the hydraulic throughput of the porous medium. The model finishes with a linear layer producing a single output, predicting the hydraulic throughput based on the arrangement of pixels in the input images. We experimented changing the activation function here as well, however, this did not lead to any significant improvements so ReLU was kept as our activation function.

Finally, the SymmetricModelWrapper class extends this architecture by hardwiring symmetry transformations into the model using specialised classes

(for instance `ApplyReflectionXSymmetry()`, `ApplyReflectionYSymmetry()`, etc). The way it works is for each forward pass, each individual input is augmented (reflected, rotated, etc). Then the input as well as all its augmentations are passed through the model. Each of the results for the throughputs are then calculated. Finally, there is an ultimate layer added to the model which calculates the average throughput predicted for the input and all its augmentations. This value is then returned as the predicted value for the input. These transformations enhance the model's robustness by exposing it to various orientations of the input images, therefore improving its ability to generalise across different viewpoints of the porous medium images. We tested models with and without symmetry, and every time symmetry was respected the model's performance drastically increased. We also experimented with omitting certain symmetries. However, that too, did not lead to any increase in accuracy.

The aspect of our model which we most regretted not experimenting with further was implementing more symmetries into the model architecture. Prior to coding we discussed the possibility of creating an algorithm which, for each image, would iterate through the (40, 40) image and recursively detect and extract every disjointed shape. It would then feed new images each containing only the disjointed shape into the model oriented in a way that would be standard across all encounters of the same shape. We would then pad everything so that the dimensions always stay the same across all inputs. The reason we theorised this symmetry would be so effective is that it would be disambiguating the input. The model would also learn to calculate throughput of shapes that are commonly repeated in the images (for example, shapes of 1 or 2 pixel size), and would therefore be able to calculate the throughput of the overall image to a extremely high degree. We were very excited at the prospect of implementing this symmetry. However, time constraint as well as computational overhead did not permit us to implement it successfully.

Computational efficiency

The architecture uses computational efficiency through its structured approach to feature extraction and dimension reduction. The convolutional layers perform the initial task of feature extraction, progressively capturing detailed representations from the input images. Following each convolutional layer, average pooling layers are used to reduce the dimensions of the inputs.

This reduction helps lower the computational overhead by decreasing the number of parameters and operations required in subsequent layers.

Although these layers lessened the computational burden, the final model we created still contained 683,000 parameters - which is larger than what we had hoped for. While developing, we created models with two different goals in tandem. The first goal was to make a model which was as accurate as possible, disregarding the number of trainable parameters for that purpose. The second was to make one which was extremely accurate with fewer than 20,000 trainable parameters. In the end, we focused more on the first goal as we needed one clear objective to provide a high quality deliverable. However, we still did a lot of exploration into models with low computational overhead.

For example, to achieve a model which retained all its symmetries hardwired into its layers and had fewer than 20,000 trainable parameters, we decided to use just 2 convolutional layers. The reasoning being that, from our understanding, each added convolutional layer theoretically extracts a more abstract feature from an image. As we thought our images were not that complex, because they contain just groups of different shapes, we thought that 2 layers would be enough to extract the most relevant features from the data. This would also significantly lower the computational burden of our model. However, after rigorous training and extensive exploration, we could not obtain satisfactory accuracies. We also tried lowering computational overhead by removing certain symmetries (specifically reflections across diagonals). However, this too decreased the accuracy of our models by a large margin. Finally, we attempted to significantly lower the number of output channels in each convolutional layer. However, this also did not result in the model performing up to our standards. In the end, we decided on focusing on making a model which was very accurate without actively accounting for the number of trainable parameters being fewer than 20,000. However, we always kept in mind to make sure it was as computationally efficient and did not use any more filters, layers, or symmetries than what was shown to be necessary through training.

Physical Properties

The `SymmetricModelWrapper` class systematically modifies each input image's symmetry by applying rotations and reflections, which enhances the network's physical properties. This method enables the network to understand

how these properties emerge across many spatial orientations by exposing it to a wider range of orientations and viewpoints of the porous medium. These changes allow the network to catch subtle variations in the physical structure and behaviour of the medium by increasing the diversity of data processed during each forward pass.

Furthermore, the fully connected layers, integral in processing the flattened feature maps, play an important role in finding complex relationships between the extracted features and the hydraulic throughput of the porous medium. These layers consolidate information from multiple perspectives—both original and augmented—to refine the model's understanding and prediction capabilities. This approach ensures that the model not only achieves accuracy in its predictions but also comprehensively reflects the authentic physical attributes and behaviours of porous materials under varying conditions and orientations.

Part F:

To prevent our network from overfitting, we employed a number of distinct methods. One such method was one of the assignment's requirements: considering symmetries. By augmenting our inputs with rotations and reflections, we exposed the model to a wider variety of examples, consequently making the model more robust to variations and distortions that it might encounter and thereby making it generalise better. We also utilised dropout layers, which help prevent overfitting by randomly deactivating a fraction of neurons during each training iteration. In this way, the network is forced to operate with a randomly chosen subset of neurons each time, preventing the neurons from co-adapting too closely to specific features of the training data.

Our two most consequential methods were hyperparameter optimization and early stopping. We used the former method all throughout the project, always trying to tweak the hyperparameters when we would get stagnant validation losses paired with decreasing training losses. Many times, the changing of parameters led to more efficient models with far more equal decreases in training losses relative to validation losses. In regards to early stopping, we used this to ensure that the model does not continue to train to the point where it starts to overfit the training data. When the model's performance on the validation set begins to degrade, it indicates that the model is starting to

learn noise and specific patterns in the training data that do not generalise well. We used this fact to prevent overfitting before it happened.

Having discussed how we tackled overfitting, we can now examine how well we did so. *Figure 21* (below) shows a visual of the training and testing losses when training our final model over each epoch. As shown, at a certain point (around epoch 1000) the training loss appears to continue decreasing steadily while the testing loss remains stagnant or decreases at a much slower rate. This is indicative of overfitting, and could have harmed the applicability of our predictions to data outside of the training set had we not taken precautions against this. While training, we realised that the training loss did decrease quicker than the testing loss (on average), and consequently implemented a feature that would take the model with the best validation loss as the final model rather than simply taking the model after the final epoch. Therefore, even if the model would be overfitting to a certain extent, as in the case shown below, we would not have to stop it as long as the validation loss would also be decreasing somewhat regularly. This was the case for our final model, where we had a somewhat consistent reduction in the validation loss despite a quicker decrease in the training loss, but was not the case for other models, which were consequently stopped prematurely.

All in all, we hypothesised that our final model's slight overfitting would not be of as great importance as long as we stored the model with the best validation loss. Nevertheless, one counterargument for this idea could be the fact that the model with the lowest validation loss could still be overfitted but, through luck, also perform well on the validation dataset. The only way we could verify this fact was through the datasets and feedback on Kaggle. Generally speaking, however, our models performed similarly on Kaggle to how they did on our validation datasets, leaving us with the conclusion that with our mitigation strategy, the impact of any overfitting was reduced. Thus, despite certain overfitting of the model in general, the impacts of this fact were minimised in a seemingly effective manner by only considering the models with the best validation loss.

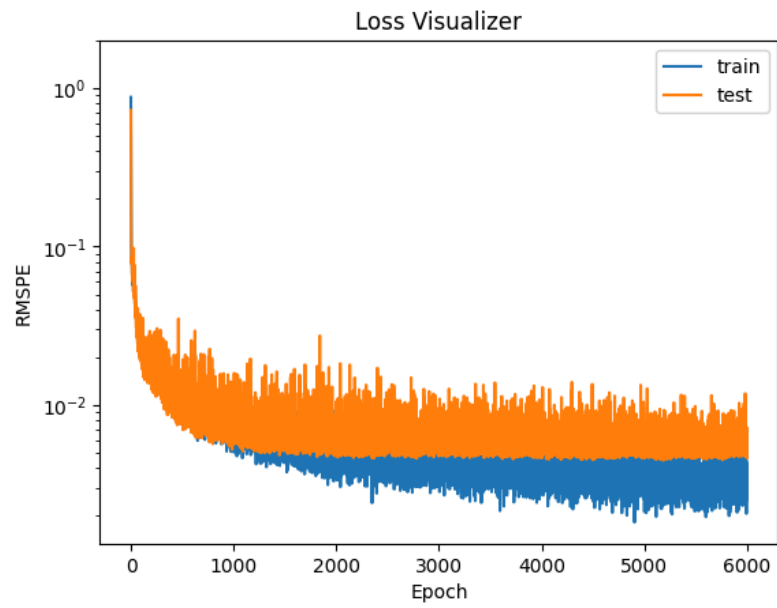


Figure 21 - Training vs Testing Loss of Final Model Over Epochs